

DESENVOLVIMENTO DE SOFTWARE PARA A WEB 1

Sistema Web: Loja / E-commerce de Produtos

Arquitetura: Modelo-Visão-Controlador (MVC)

Atividade AA-1: Sistema para uma loja virtual (e-commerce)

O sistema deve possuir um cadastro de **clientes**, com os seguintes dados: e-mail, senha, CPF, nome, telefone, sexo e data de nascimento.

O sistema deve possuir um **administrador** (operador da loja), responsável pela gestão do catálogo e dos pedidos, com os seguintes dados de acesso: e-mail (ou nome de usuário) e senha.²

O catálogo da loja é organizado em **categorias**, e cada **produto** pertence a uma categoria. As compras dos clientes são registradas como **pedidos**, e cada pedido é composto por um ou mais **itens de pedido** (produto + quantidade).

O sistema deve atender aos seguintes requisitos:

- **R1:** CRUD¹ de **clientes** (requer login de administrador).
- **R2:** CRUD de **categorias** de produtos (requer login de administrador). Os dados de uma categoria são: nome e (opcionalmente) descrição.
- **R3:** CRUD de **produtos** (requer login de administrador). Depois de fazer login, o administrador pode cadastrar, editar e remover produtos do catálogo. O cadastro de produtos deve possuir os seguintes dados: nome, descrição, preço, quantidade em estoque, categoria à qual pertence e fotos (no máximo 10 imagens) do produto.
- **R4:** Listagem de todos os produtos em uma única página (não requer login). O sistema deve prover a funcionalidade de **filtrar** os produtos por nome, por categoria ou por faixa de preço.
- **R5:** Realização de **pedido de compra** (requer login do cliente via e-mail + senha). Ao navegar pelo catálogo (requisito R4), o cliente pode adicionar produtos ao seu pedido (carrinho), informando a quantidade de cada item, e finalizar a compra. A data atual (quando o pedido foi realizado) deve ser registrada no sistema, e o pedido nasce com o status **ABERTO**. O cliente deve ser informado (via e-mail) sobre a confirmação do pedido, com o resumo dos itens e o valor total.
- **R6:** Listagem de todos os **pedidos de um cliente** (requer login do cliente via e-mail + senha). Depois de fazer login, o cliente pode visualizar todos os seus pedidos com os respectivos *status* e o valor total de cada um. **ABERTO** indica que o pedido foi realizado e aguarda processamento. **PAGO** indica que o pagamento foi confirmado. **ENVIADO** indica que o pedido foi despachado. **CANCELADO** indica que o pedido foi cancelado.
- **R7:** Gestão de **pedidos** pelo administrador (requer login de administrador). Depois de fazer login, o administrador pode visualizar todos os pedidos realizados na loja e **atualizar o status** de cada um (**ABERTO** → **PAGO** → **ENVIADO**, ou **CANCELADO**). A cada mudança de status, o cliente deve ser informado (via e-mail). O sistema deve também prover a funcionalidade de filtrar os pedidos por status.
- **R8:** O sistema deve ser **internacionalizado** em pelo menos dois idiomas: português + outro de sua escolha.

- **R9:** O sistema deve **validar** (tamanho, formato, etc.) todas as informações (campos nos formulários) cadastradas e/ou editadas.

O sistema deve **tratar todos os erros possíveis** (cadastros duplicados, problemas técnicos, etc.) mostrando uma página de erros amigável ao usuário e registrando o erro no console.

Arquitetura

Modelo-Visão-Controlador (MVC).

Tecnologias

- Spring MVC, Spring Data JPA, Spring Security & Thymeleaf (Lado Servidor)
- JavaScript & CSS (Lado Cliente)

Ambiente de Desenvolvimento

- A compilação e o *deployment* devem ser obrigatoriamente realizados via **Maven**.
- Os arquivos-fonte do sistema devem estar hospedados obrigatoriamente em um repositório (preferencialmente **GitHub**).

1. CRUD: Create, Read, Update & Delete.

2. O login + senha do administrador devem ser populados no banco de dados durante a inicialização do sistema.

DESENVOLVIMENTO DE SOFTWARE PARA A WEB 1

Sistema Web: Loja / E-commerce de Produtos

Atividade AA-2: Sistema para uma loja virtual (e-commerce) — REST API

Obs 1: Esta atividade deve ser baseada na atividade AA-1. Ou seja, deve-se apenas implementar os novos requisitos (funcionalidades providas em uma REST API) aqui mencionados — levando em consideração o que já foi desenvolvido na atividade AA-1.

O sistema deve incorporar os seguintes requisitos.

REST API — CRUD de clientes

- **Cria um novo cliente [Create - CRUD]**
POST <http://localhost:8080/api/clientes>
Body: raw/JSON (application/json)
- **Retorna a lista de clientes [Read - CRUD]**
GET <http://localhost:8080/api/clientes>
- **Retorna o cliente de id = {id} [Read - CRUD]**
GET <http://localhost:8080/api/clientes/{id}>
- **Atualiza o cliente de id = {id} [Update - CRUD]**
PUT <http://localhost:8080/api/clientes/{id}>
Body: raw/JSON (application/json)
- **Remove o cliente de id = {id} [Delete - CRUD]**
DELETE <http://localhost:8080/api/clientes/{id}>

REST API — CRUD de categorias

- **Cria uma nova categoria [Create - CRUD]**
POST <http://localhost:8080/api/categorias>
Body: raw/JSON (application/json)
- **Retorna a lista de categorias [Read - CRUD]**
GET <http://localhost:8080/api/categorias>
- **Retorna a categoria de id = {id} [Read - CRUD]**
GET <http://localhost:8080/api/categorias/{id}>
- **Atualiza a categoria de id = {id} [Update - CRUD]**
PUT <http://localhost:8080/api/categorias/{id}>
Body: raw/JSON (application/json)
- **Remove a categoria de id = {id} [Delete - CRUD]**
DELETE <http://localhost:8080/api/categorias/{id}>

REST API — CRUD de produtos

- **Cria um novo produto [Create - CRUD]**
POST <http://localhost:8080/api/produtos>
Body: raw/JSON (application/json)
- **Retorna a lista de produtos [Read - CRUD]**
GET <http://localhost:8080/api/produtos>
- **Retorna o produto de id = {id} [Read - CRUD]**
GET <http://localhost:8080/api/produtos/{id}>
- **Atualiza o produto de id = {id} [Update - CRUD]**
PUT <http://localhost:8080/api/produtos/{id}>
Body: raw/JSON (application/json)
- **Remove o produto de id = {id} [Delete - CRUD]**
DELETE <http://localhost:8080/api/produtos/{id}>

REST API — Consultas adicionais

- **Retorna a lista de produtos cujo nome contém {nome} [Read - CRUD]**
GET <http://localhost:8080/api/produtos?nome={nome}>
- **Retorna a lista de produtos da categoria de id = {id} [Read - CRUD]**
GET <http://localhost:8080/api/produtos/categorias/{id}>
- **Cria um novo pedido para o cliente de id = {id} [Create - CRUD]**
POST <http://localhost:8080/api/pedidos/clientes/{id}>
Body: raw/JSON (application/json)
- **Retorna a lista de pedidos do cliente de id = {id} [Read - CRUD]**
GET <http://localhost:8080/api/pedidos/clientes/{id}>
- **Retorna a lista de pedidos por status = {status} [Read - CRUD]**
GET <http://localhost:8080/api/pedidos/status/{status}>

Obs 2: Em todas as funcionalidades da REST API mencionadas acima, não há necessidade de autenticação (login).

Dica: Na configuração do Spring Security, utilize algo semelhante ao apresentado no código abaixo (liberando os endpoints `/api/**` e exigindo login nas demais rotas Web):

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests((authz) -> authz
        // Endpoints da REST API (publicos)
        .requestMatchers("/api/**").permitAll()
        // Paginas publicas (catalogo, login, recursos estaticos)
        .requestMatchers("/", "/produtos", "/login", "/css/**", "/js/**").permitAll()
        // Rotas de administracao exigem o papel ADMIN
        .requestMatchers("/admin/**").hasRole("ADMIN")
        // Demais rotas exigem usuario autenticado (cliente logado)
        .anyRequest().authenticated()
        .csrf(AbstractHttpConfigurer::disable)
        .formLogin((form) -> form
            .loginPage("/login")
            .permitAll())
        .logout((logout) -> logout
            .logoutSuccessUrl("/").permitAll());
    return http.build();
}
```

Arquitetura

Modelo-Visão-Controlador (MVC).

Tecnologias

- Spring MVC (Controladores REST), Spring Data JPA, Spring Security & Thymeleaf (Lado Servidor)

Ambiente de Desenvolvimento

- A compilação e o *deployment* devem ser obrigatoriamente realizados via **Maven**.
- Os arquivos-fonte do sistema devem estar hospedados obrigatoriamente em um repositório (preferencialmente **GitHub**).

1. CRUD: Create, Read, Update & Delete.